

Projet final : Technique de Simulation

G10 - M1 Actuariat I.S.F.A. 2024/2025

Auteurs : Baptiste Plantier & Adel Bettache

Résumé

On considère un porte-feuille de N contrats d'assurance chacun associé à un motard.

On considère que la distribution d'un remboursement est proportionnelle à

$$f^*(x) = \exp(-\eta \ln(\alpha + |x - x_0|)) \mathbb{1}_{x \in \mathbb{R}^+}$$
$$\eta > 3, x_0 > 0, \alpha > 0$$

De plus, on considère que les conditions météorologiques ont un impact sur les risques encourus par les motards. La météo sera modélisée par une chaîne de Markov à trois états, *Soleil*, *Nuages*, *Pluies*. Avec pour probabilité de transition un vecteur

$$\text{ptrans} = (p_{SN}, p_{NS}, p_{NP}, p_{PN})$$

Chaque motard a une probabilité p_S (resp. p_N , resp. p_P) d'avoir un accident s'il se trouve dans l'état S (resp. N , resp. P). Ces probabilités sont contenues dans le vecteur

$$\text{pacc} = (p_S, p_N, p_P)$$

Nous allons nous placer dans deux cadres distincts.

Cas A Tous les motards ont la même météo.

Cas B Chaque motard a sa propre météo (*i.e.* sa propre chaîne de Markov) et elles sont toutes indépendantes.

L'objectif sera d'estimer par Monte-Carlo la quantité

$$m(s) := \mathbb{E}[R - s \mid R > s] \tag{1}$$

Où R est le montant des remboursements totaux sur une année et $s > 0$ un seuil fixé à l'avance. Ceci dans le cas **A** comme le cas **B**.

Table des matières

1	Organisation du travail de groupe	2
1.1	Composition	2
1.2	Répartition des tâches : le programme	2
1.3	Répartition des tâches : le rapport	3
1.4	Outils employés	3
2	Choix des paramètres.	3
2.1	Analyses et interprétations des paramètres.	3
2.2	Choix des paramètres de test	6
2.2.1	Choix des paramètres de remboursement	6
2.2.2	Choix des paramètres de météo	7
2.2.3	Choix des paramètres d'accidents	9
2.2.4	Choix des paramètres de seuil et nombre de motards . . .	10
3	Simulation des remboursements et de la météo.	10
3.1	Calculs explicites	10
3.2	Méthode par inversion de la fonction de répartition.	12
3.3	Comparaison avec une loi de Cauchy.	14
3.4	Simulation de la météo.	16
4	Calcul de $m(s)$ sous A et B.	18
4.1	Monte-Carlo simple.	18
4.2	Monte-Carlo avec réduction de variance.	20
4.3	Implémentation de <code>msb.c</code>	22
5	Pistes d'améliorations.	25
5.1	Le modèle.	25
5.2	Le programme.	25

1 Organisation du travail de groupe

1.1 Composition

Notre groupe est composé par :

- Baptiste Plantier
- Adel Bettache

1.2 Répartition des tâches : le programme

Pour la répartition des tâches, Baptiste a fait toutes les recherches qui permettaient de justifier des paramètres employés dans le code, il a aussi créé la première version de la fonction *markov*, une version initiale de *remb* et de *msa*.

Adel quant à lui s'est chargé du code final des fonctions utilisés dans le rapport (*remb*, *msa*, *msb*, *markov* etc.), l'implémentation en C de *msb* et de la réduction

de variance.

À noter que le groupe s'est composé assez tardivement et que l'on avait tous les deux des versions différentes de `remb`, `msa` et `markov` ce qui nous a permis de sélectionner les plus performantes et d'améliorer celles qui coïncidaient.

1.3 Répartition des tâches : le rapport

Pour la rédaction du rapport, Baptiste s'est chargé de la partie 1, 2 et 5 et Adel s'est chargé de la partie 3, 4 et 5.

1.4 Outils employés

Pour la rédaction du rapport en LaTeX, nous avons utilisé l'interface collaborative d'Overleaf. Et pour les codes nous avons utilisé l'IDE R-Studio et nous partageons nos codes via la plateforme *codeshare*.

2 Choix des paramètres.

2.1 Analyses et interprétations des paramètres.

Avant même de commencer à choisir des valeurs pour les paramètres, il faut calculer le coefficient de normalisation pour obtenir une densité. Une idée serait d'utiliser la fonction *integrate* incluse dans R or ceci ne fonctionne pas dans tous les cas. En effet pour certaines valeurs des paramètres, la fonction renvoie une erreur. Car lorsque l'on regarde l'allure de la fonction f^* on se rend compte que pour certaines valeurs des paramètres la fonction est un pique et ceci est très mal géré par la fonction *integrate* qui renvoie alors une erreur.

Le calcul direct de l'intégrale se fait assez aisément.

$$\begin{aligned}\int_{-\infty}^{+\infty} f^*(x)dx &= \int_0^{+\infty} \exp(-\eta \ln(\alpha + |x - x_0|))dx \\ &= \int_0^{x_0} \exp(-\eta \ln(\alpha + x_0 - x))dx + \int_{x_0}^{+\infty} \exp(-\eta \ln(\alpha + x_0 - x))dx \\ &= \int_0^{x_0} \frac{dx}{(\alpha + x_0 - x)^\eta} + \int_{x_0}^{+\infty} \frac{dx}{(\alpha + x - x_0)^\eta}\end{aligned}$$

Pour la première intégrale (resp. la seconde) on peut faire le changement de variable

$$y = \alpha + x_0 - x \quad (\text{resp. } y = \alpha + x - x_0)$$

Et on obtient que le coefficient de normalisation c vaut

$$c = \frac{1 - \eta}{(\alpha + x_0)^{1-\eta} - 2\alpha^{1-\eta}}$$

Ainsi la fonction $f : x \mapsto cf^*(x)$ est bien une densité.

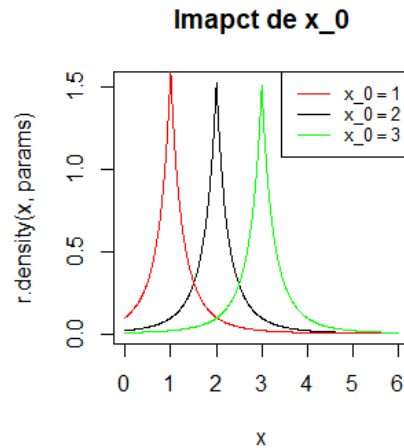
On la met dans notre code car elle nous permettra de faire des vérifications.

```
r.density <- function(x, params) {
  c <- (1-params$eta)/
    ((params$alpha+params$x0)^(1-params$eta)
     -2*params$alpha^(1-params$eta))
  ifelse(test = x < 0, 0, c*exp(- params$eta *
    log(params$alpha + abs(x - params$x0))))
}
```

Maintenant que l'on a la densité d'un remboursement, on peut essayer de voir l'influence des paramètres α, η, x_0 sur la densité.

Influence des paramètres sur la forme de la densité

Influence de x_0 On fixe $\alpha = 1, \eta = 4$ et faisons varier x_0 afin d'essayer d'en tirer une information.



Très clairement, on voit que x_0 a tendance à donner l'endroit où se concentrent les montant des remboursements.

Cela était prévisible puisque la densité atteint son maximum en x_0 . En effet on a

$$f(x) = \exp(-\eta \ln(\alpha + |x - x_0|)) 1_{x \in \mathbb{R}^+}$$

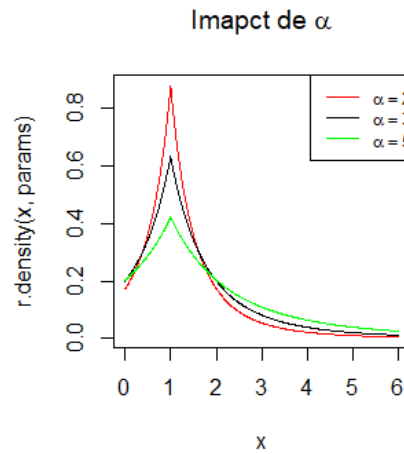
afin de maximiser f il faut minimiser

$$x \mapsto \eta \log(\alpha + |x - x_0|)$$

La solution à ce problème est évidemment $x = x_0$.

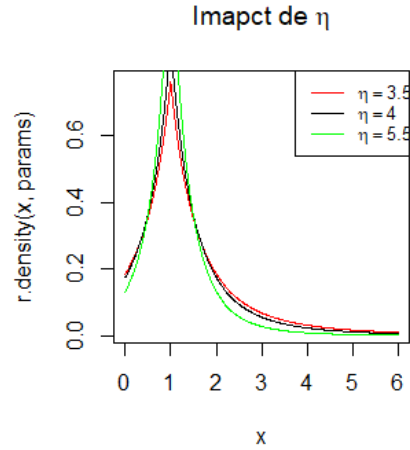
Ainsi x_0 représente le remboursement le plus probable.

Influence de α On fixe $x_0 = 1$, $\eta = 4$ et faisons varier α .

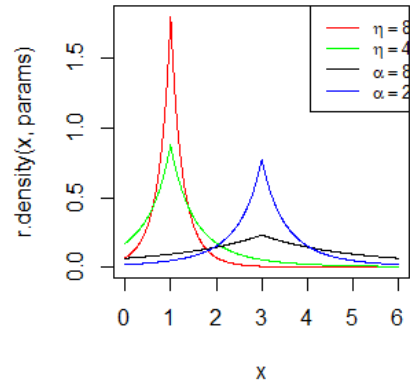


On voit bien graphiquement que plus α augmente plus la densité *s'aplatit* et donc on a bien plus de chance de s'éloigner de x_0 . Le fait que α aplatisse la distribution est visible directement dans la formule de la densité. L'influence de α sur la densité n'a donc rien de très surprenant.

Influence de η On fixe $x_0 = 1$, $\alpha = 2$ et on fait varier η .



Tout comme α , η joue sur la dispersion des remboursements autour de x_0 mais η a une influence bien plus forte que celle de α sur la dispersion des valeurs autour de x_0 , en voici une illustration.



2.2 Choix des paramètres de test

2.2.1 Choix des paramètres de remboursement

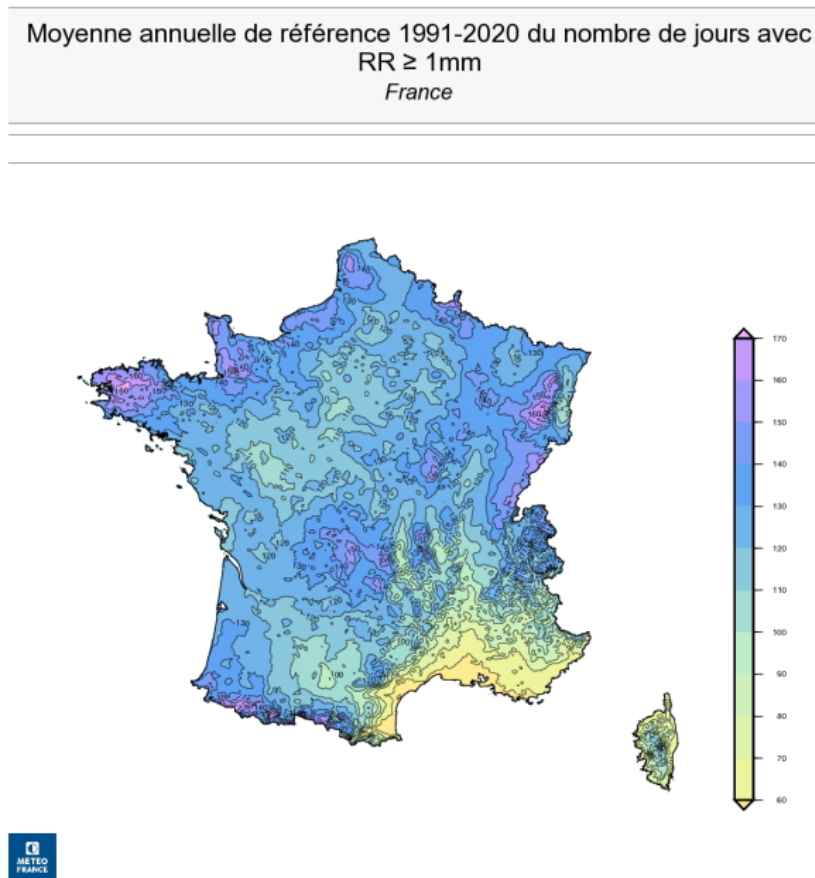
Afin de simuler la densité d'un remboursement nous avons besoin des paramètres de la densité (α, x_0, η) . Cependant, la loi d'un remboursement est donné dans une unité arbitraire, nous ne pouvons donc pas chercher à ce que ces paramètres collent la réalité. Nous prendrons donc dans la suite les paramètres suivants :

```
params <-list(alpha = 1, x0 = 2, eta = 4)
```

2.2.2 Choix des paramètres de météo

Pour simuler la météo, on utilise une chaîne de Markov à trois états. "Soleil, Nuage, Pluie". Toutes les transitions sont possibles sauf "Pluie" à "Soleil" et "Soleil" à "Pluie". On veut donc trouver des probabilités de transition telle que la chaîne de Markov simulée reflète la météo en France sur une année. Les informations que nous utiliserons proviennent du site de MétéoFrance [1].

Commençons par les jours de pluie : on utilisera la définition de météoFrance "un jour est considéré pluvieux si les précipitations sont supérieur à 1 mm (c'est à dire 1L/m²)". La carte suivante donne le nombre de jours de pluie moyen sur une année entre 1991 et 2020 :



Le nombre maximal de jours de pluie est 170 et le minimum est 50. Le nombre

de jours de pluie semble être en moyenne 110 jours, cependant les régions avec peu de pluie ne représentent que le sud-est de la France et une grande partie du territoire se trouve au-dessus de 120 jours de pluie annuels. On considérera donc 120 jours de pluie annuels en moyenne.

Pour les jours ensoleillés et nuageux, on considère qu'un jour est ou bien entièrement ensoleillé ou bien entièrement nuageux. En France il y a en moyenne 2200 heures d'ensoleillement annuelle [1] [2] et le jour dur en moyenne 12h on est déduit que pour avoir l'ensoleillement requis, il doit y avoir $2200/12$ jours ensoleillé que nous arrondirons à 184 jours ensoleillés par an.

Ainsi on cherche des probabilités de transition telles qu'il y ait en moyenne 120 jours de pluie, 184 jours de soleil et donc $365 - (120 + 184) = 61$ jours nuageux par an. En prenant les probabilités de transitions :

$$p_{SN} = 0.172 \quad p_{NS} = 0.51 \quad p_{NP} = 0.33 \quad p_{PN} = 0.165$$

On obtient une chaîne de Markov proche de ce que l'on cherchait. On vérifie avec un programme qui simule une chaîne de Markov (on améliorera ce programme par la suite pour le calcul de $m(s)$) :

```
markov <- function(N,ptrans){
  P<-matrix(0,nrow=3,ncol=3)
  P[1,1]<-1-ptans[1]
  P[1,2]<-ptans[1]
  P[2,1]<-ptans[2]
  P[2,2]<-1-ptans[2]-ptans[3]
  P[2,3]<-ptans[3]
  P[3,2]<-ptans[4]
  P[3,3]<-1-ptans[4]
  s<-1
  v<-numeric(N)
  v[1]<-1
  for(k in 2:N){
    s<- v[k-1]
    v[k]<-sample(1:3,1,prob = P[s,])
  }
  return(v)
}
ptrans=c(0.172,0.51,0.33,0.165)
M<-replicate(10000,markov(365,ptrans))
h<-c(sum(M==1),sum(M==2),sum(M==3))/10000
h
```

Qui renvoie le vecteur (183.7707,61.0865,120.1428) où la première coordonnée (resp. seconde, troisième) est le nombre de jours de soleil (resp. nuageux, pluvieux) sur une année moyenné sur 10000 ans. On a donc bien une répartition proche de ce que l'on cherchait pour la suite, on considérera donc pour les tests :

```
ptrans <- (0.172,~0.51,~0.33,~0.165)
```


2.2.3 Choix des paramètres d'accidents

Afin de mener à bien les simulations, nous avons besoin des probabilités d'accidents pour chaque motard et pour chaque météo (Pluie, Nuageux, Ensoleillé).

D'après le site du gouvernement chargé du développement durable [3], il y a environ 2.3 millions de motards en France et environ 60% circulent les jours de semaine, et moins le week-end. On fera donc l'approximation qu'il y a $2300000 \times 0.6 = 1.3$ millions de motards sur les routes chaque jour. De plus toujours d'après le même site [4], il y a environ 47 000 000 de véhicules en circulation en France dont 73% sont utilisés pour un trajet quotidien. D'après le site de la tribune auto [5] les véhicules sont moins utilisés le week-end, on pourra donc faire l'approximation qu'il y a $0.73 \times 47 = 34$ millions de véhicules en circulation chaque jour en France. Ainsi, les motards représentent environ 4% du trafic journalier.

D'après le site du journal Lefigaro [6] il y a chaque année 1.9 million de constats amiables effectués en France et environ 50 000 accidents corporels soit environ 2 millions d'accidents déclarés aux assurances. Les motards représentant 4% de la circulation, on peut faire l'approximation que 80 000 constats amiables et accidents concernent des motards chaque année. Ainsi, on peut estimer qu'il y aurait $\frac{80000}{1300000} = 0.062$ accidents par motard chaque année. C'est à dire pour 1000 motards environ 62 accidents par an. En prenant comme probabilité d'accident :

$$p_S = 0.0001 \quad p_N = 0.00015 \quad p_P = 0.0003$$

On obtient un nombre d'accidents proche de ce que l'on souhaite. On vérifie avec le code suivant :

```
acc<-function(M,N,pacc){
  s<-sum(M == 1)
  p<-sum(M == 3)
  n<-sum(M == 2)
  as<-sum(sample(0:1,s*N,prob=c(1-pacc[1],pacc[1]),
                                     replace=T))
  an<-sum(sample(0:1,n*N,prob=c(1-pacc[2],pacc[2]),
                                     replace=T))
  ap<-sum(sample(0:1,p*N,prob=c(1-pacc[3],pacc[3]),
                                     replace=T))
  a <- as + an + ap
  a
}
mean(replicate(10000,acc(markov(365,params$ptrans),
                           1000,params$pacc)))
[1] 62.5471
```

Où les probabilités ptrans sont celle trouvées précédemment et la fonction markov est celle décrite dans la partie précédente. On voit qu'avec ces paramètres, on est proche du nombre d'accidents voulu, on considérera donc pour nos tests :

```
pacc=c(0.0001,0.00015,0.0003)
```

2.2.4 Choix des paramètres de seuil et nombre de motards

Pour le choix du nombre de motards, on se basera sur la statistique précédente [3], il y a 2.3 millions de motards en France et on s'intéresse dans le cadre de l'hypothèse A à une compagnie d'assurance régionale et dans l'hypothèse B, le nombre de motards doit être limité pour que chacun puisse avoir sa propre région. On considérera donc que l'on s'intéresse à 1% du parc actif, c'est à dire environ 13 000 motards comme vu dans la partie précédente. On conduira donc les tests avec 13 000 motards.

Pour le choix du seuil, regarde d'abord la répartition des remboursements sur un an pour 13 000 motards et les paramètres choisis plus tôt en utilisant les fonctions markov et acc définies plus tôt :

```
Remba<-function(params){
  M<-markov(365,params$ptrans)
  nacc<- acc(M,params$N,params$pacc)
  R<-sum(rremb(nacc,params))
}
quantile(replicate(2000,Remba(params)),c(0.10,0.25,0.50,0.75,0.90,1))
      10%      25%      50%      75%      90%      100%
1418.146 1528.264 1649.189 1785.420 1903.745 5659.706
```

Ainsi 90% des remboursements annuels sont en dessous de 1 900, nous utiliserons donc 2000 comme seuils afin de vraiment capter les valeurs élevées des remboursements. Dans la suite, nous utiliserons donc :

```
N = 13000, s = 2000
```

3 Simulation des remboursements et de la météo.

3.1 Calculs explicites

Nous avons la densité et nous adopterons la méthode par inversion de la fonction de répartition.

Afin de trouver la fonction de répartition F_r , on calcul

$$F_r(x) = \int_{-\infty}^x f_r(s)ds$$

Après calcul, on trouve :

$$F_r(x) = \begin{cases} 0 & \text{si } x \leq 0 \\ c \left(\frac{(\alpha+x_0)^{1-\eta}}{1-\eta} - \frac{(\alpha+x_0-x)^{1-\eta}}{1-\eta} \right) & \text{si } 0 \leq x \leq x_0 \\ c \left(\frac{(\alpha-x_0+x)^{1-\eta}}{1-\eta} \right) + 1 & \text{si } x \geq x_0 \end{cases}$$

On peut vérifier que la fonction est bien continue, qu'elle vaut 0 en $-\infty$ et qu'elle tend vers 1 quand $x \rightarrow +\infty$.

Après inversion de la fonction on trouve $G :]0, 1[\rightarrow \mathbb{R}^+$ et $\forall y \in]0, 1[$ on a :

$$G(y) = \begin{cases} \alpha + x_0 - \exp\left(\frac{1}{1-\eta} \log\left((\alpha + x_0)^{1-\eta} - \frac{(1-\eta)}{c} y\right)\right) & \text{si } 0 < y < F(x_0) \\ x_0 - \alpha + \exp\left(\frac{1}{1-\eta} \log\left(\frac{(1-\eta)}{c} (y - 1)\right)\right) & \text{si } F(x_0) \leq y < 1 \end{cases}$$

On code la fonction de répartition ainsi que sa réciproque afin d'effectuer des vérifications qui conforteraient le fait d'avoir les bonnes fonctions.

```
r.repartition <- function(x, params) {
  if(x < 0) {
    return(0)
  }
  else if(x < params$x0) {
    return(c*((params$alpha + params$x0)^(1 - params$eta) -
      (params$alpha + params$x0 - x)^(1 - params$eta))
      /((1 - params$eta)))
  }
  else {
    return(c*((params$alpha - params$x0 + x)^(1 - params$eta)
      /((1 - params$eta)) + 1)
  }
}

r.inv_repartition <- function(y, params) {
  F_0 <- r.repartition(0, params)
  F_x0 <- r.repartition(params$x0, params)

  result <- numeric(length(y))
  result[y < F_0] <- 0

  cond1 <- (y > F_0 & y < F_x0)
  if (any(cond1)) {
    result[cond1] <- params$alpha + params$x0 -
      exp((1/(1 - params$eta)) *
        log((params$alpha + params$x0)^(1 - params$eta) -
          ((1 - params$eta) / c) * y[cond1]))
  }
  cond2 <- (y >= F_x0)
  if (any(cond2)) {
    result[cond2] <- params$x0 - params$alpha +
      exp((1/(1 - params$eta)) * log((1 - params$eta)
        / c * (y[cond2] - 1)))
  }
  return(result)
}
```

Première vérification :

```
x <- seq(0, 10, 0.1)
u <- sapply(x, function(z) (r.repartition(z, params))
print(r.inv_repartition(u, params))
```

Ce code affiche les valeurs exactes du vecteur x , ce qui signifie que `r.inv_repartition` inverse correctement `r.repartition`.

Afin de vérifier que `r.repartition` correspond bien à nos attentes, on peut la comparer avec l'approximation que peut en faire R.

```
approx_repartition <- function(x, params) {
  sapply(x, function(xi) {
    integrate(function(t) (r.density(t, params)),
      lower = -Inf, upper = xi)$value
  })
}

x <- seq(0, 10, 0.01)
u <- sapply(x, function(t) (r.repartition(t, params)))
mse <- (1/length(x))*sum(approx_repartition(x, params)-u)^2
print(mse)
```

Avec une valeur d'environ 3×10^{-8} , on peut sans difficulté considérer que notre fonction de répartition est la bonne.

3.2 Méthode par inversion de la fonction de répartition.

Le principe est simple :

- On simule y selon une loi uniforme sur $[0, 1]$
- On applique la fonction inverse à y
- On renvoie ce résultat

```
rremb <- function(n, params) {
  y <- runif(n) #Premiere etape

  # Pre calcul de certaines constantes recurrentes
  a <- params$alpha + params$x0
  a_moins <- params$x0 - params$alpha
  e <- 1 - params$eta
  inv_e <- 1/e
  a_e <- a^e
  c <- e/(a_e - 2 * params$alpha^e)
  e_c <- e/c
  F_x0 <- c*((params$alpha)^(e)/(e)) + 1
  cond1 <- y < F_x0
  cond2 <- !cond1

  # Application de la fonction de repartition inverse
  y[cond1] <- a - exp(inv_e*log(a_e - e_c*y[cond1]))
  y[cond2] <- a_moins + exp(inv_e*log(e_c*(y[cond2] - 1)))
```

```

    #On renvoie le resultat
    return(y)
}

```

Afin de vérifier le calcul, on peut comparer cela aux résultats théoriques de la moyenne et de la variance dont voici les formules :

```

e <- c*(((params$alpha + params$x0)/(1 - params$eta))*
((params$alpha + params$x0)^(1-params$eta) -
params$alpha^(1 - params$eta) ) - (1/(2-params$eta))*
( (params$alpha + params$x0)^(2-params$eta) -
params$alpha^(2-params$eta) ) + ((params$alpha - params$x0)/
(1-params$eta))*
(params$alpha)^(1-params$eta) -
(params$alpha^(2-params$eta))/(2-params$eta))

v <- c*(((params$alpha + params$x0)^2)/(1-params$eta))*
((params$alpha + params$x0)^(1-params$eta)-
params$alpha^(1-params$eta))-((2*(params$alpha + params$x0))
/(2-params$eta))*((params$alpha + params$x0)^(2-params$eta)
-params$alpha^(2-params$eta))+(1/(3-params$eta))*
((params$alpha + params$x0)^(3-params$eta)-
params$alpha^(3-params$eta))+
((- (params$alpha - params$x0)^2)/(1-params$eta))*
(params$alpha)^(1-params$eta)+
((2*(params$alpha - params$x0))/(2-params$eta))*
params$alpha^(2-params$eta)-(1/(3-params$eta))*
params$alpha^(3-params$eta)) - e^2

```

Afin de vérifier si notre `rremb` fonctionne bien, on peut comparer la moyenne et la variance obtenue sur un grand nombre de tirages en effectuant le code suivant :

```

y_e <- replicate(100, mean(rremb(10^6, params)))
y_v <- replicate(100, var(rremb(10^6, params)))

mse_E <- (1/100)*sum( (y_e - e)^2 )
mse_V <- (1/100)*sum( (y_v - v)^2 )

```

Et on trouve $mse_E = 5,9 \times 10^{-7}$ et $mse_V = 6,4 \times 10^{-4}$ qui sont des résultats tout à fait satisfaisants.

En terme de rapidité d'exécution on obtient un temps moyen d'exécution de 0.11 seconde pour simuler 10^6 valeurs.

On peut également tracer un histogramme des valeurs générées ainsi que la densité afin de vérifier que les valeurs collent bien. On utilise le code suivant :

```

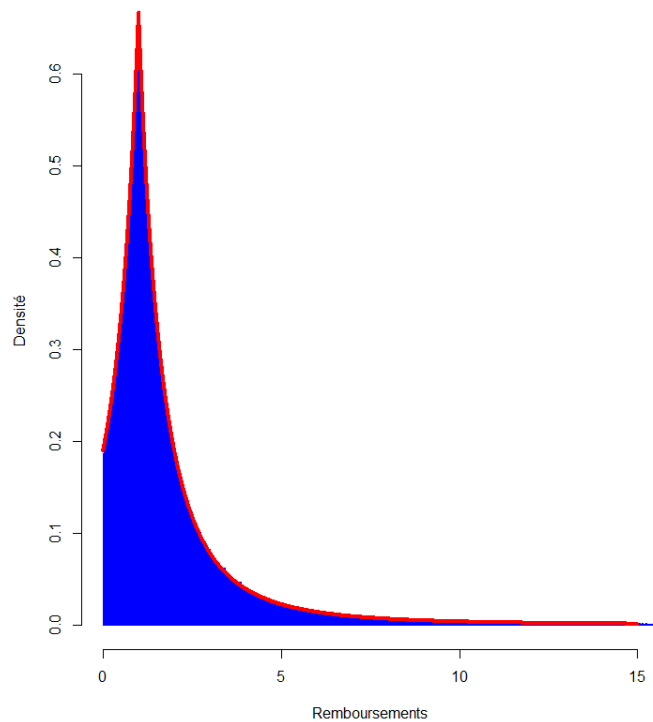
fnorm <- function(x,alpha,x0,eta){v<-(eta-1)
(1/((2/(v*alpha**v))-(1/(v*(alpha+x0)**v))))*exp(-eta*log(alpha+abs(x-x0)))
# Densite d'un remboursement

```

```

hist(rremb(1000000,params),probability = TRUE,col='blue',border = "blue",
breaks="FD", xlim=c(0,15),main="",ylab = "Densite",xlab="Remboursements")
# On trace un histogramme, la majorite des valeurs etant en dessous de
15,
on tronque afin de voir un histogramme plus propre.
x=seq(0,15,by=0.001)
lines(x,fnorm(x,params$alpha,params$x0,params$eta),col="red",lwd = 4)
# On trace la densite sur [0,15]

```



On voit que la densité colle bien l'histogramme, nos simulations semblent donc très satisfaisantes.

3.3 Comparaison avec une loi de Cauchy.

On pourrait aussi essayer de majorer la densité par une autre densité, plus simple à calculer. Comme le support de la densité est $\mathbb{R}^+$, on ne peut pas prendre une loi uniforme sauf à tronquée la densité f mais cela ne semble pas légitime car on ferait mécaniquement baisser la probabilité d'occurrences d'événements rares.

Au vu de ce à quoi ressemble la densité, une idée naturelle serait de la comparer à

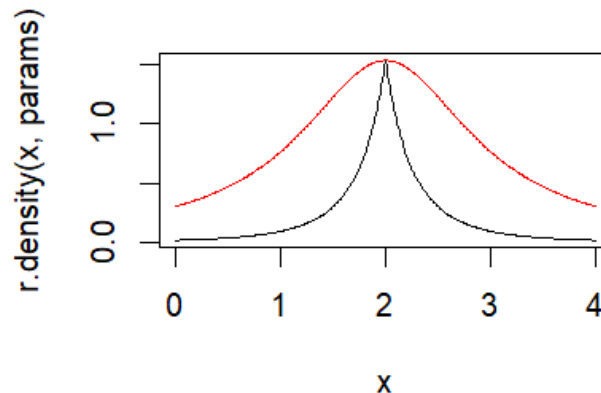
une loi normale, mais ça coince car la Gaussienne est à décroissance exponentielle et pas notre densité, ainsi il y a **nécessairement** un $x \in \mathbb{R}^+$; $f(x) > \varphi(x)$ avec φ la densité d'une loi normale de n'importe quels paramètres μ, σ^2 .

D'où le fait que le champ de recherche d'une densité majorant la nôtre se restreint à celle ayant des queues lourdes. Celle que nous avons choisi est la densité de Cauchy.

```
k <- r.density(params$x0, params)
      /dcauchy(params$x0, params$x0)
x <- seq(0, 4, 0.01)

plot(x, r.density(x, params), type = 'l')
lines(x, k*dcauchy(x, params$x0), col = 'red')
```

Ce code renvoie l'illustration suivante qui confirme, au moins visuellement, que la densité de Cauchy est *proche* de f et majore bien notre densité (au moins sur l'intervalle considéré).



On peut aussi vérifier empiriquement que la densité f est toujours en dessous de la densité de Cauchy avec le code suivant :

```
aire_f <- integrate(function(q) (r.density(q, params)),
                    0, 1000)$value
aire_cauchy <- pcauchy(1000, params$x0)

x <- seq(0, 1000, 0.1)
u <- sapply(x, function(t) (r.density(t, params)))
v <- k*dcauchy(x, params$x0)

print(any(u > v))
```

Ce code renvoie **FALSE**, ainsi sur l'intervalle $[0, 1000]$, il n'y a aucun moment où la densité passe au dessus de la densité de Cauchy.

De plus, on a $\text{aire}_f = 0.999997$ et $\text{aire}_{\text{cauchy}} = 0.9997$ d'où le fait que sur cet intervalle, on capte l'immense majorité des valeurs prise par les densités. Le choix de la densité de Cauchy est donc légitime.

On peut donc appliquer notre algorithme pour simuler la loi d'un remboursement selon la méthode de la comparaison.

```
rremb_comp <- function(n, params) {
  k <- r.density(params$x0, params)
              /dcauchy(params$x0, params$x0)
  affaire <- rep(TRUE, n)
  x <- rep(NA, n)
  while(any(affaire)) {
    n <- sum(affaire)
    x[affaire] <- rcauchy(n, params$x0)
    y <- runif(n, 0, k)
    affaire[affaire] <- y*dcauchy(x[affaire], params$x0)
                          > r.density(x[affaire], params)
  }
  x
}
```

En effectuant les mêmes vérifications empiriques, on trouve que cette fonction convient elle aussi mais elle est environ 6 fois plus lente que la précédente.

Nous garderons ainsi la méthode par inversion de la fonction de répartition.

3.4 Simulation de la météo.

La méthode *classique* de simulation d'une chaîne de Markov a été donnée dans la partie sur les choix des paramètres de transitions mais ce code peut être améliorer.

Plutôt que d'utiliser la fonction *sample*, on peut plutôt travailler sur les probabilités cumulées.

```
markov <- function(n, params) {
  etats <- c('S', 'N', 'P')
  P <- matrix(c(1 - params$ptrans[1], params$ptrans[1], 0,
                params$ptrans[2],
                1 - (params$ptrans[2] + params$ptrans[3]),
                params$ptrans[3],
                0, params$ptrans[4], 1 - params$ptrans[4]),
              nrow = 3, byrow = TRUE)
  P_cum <- t(apply(P, 1, cumsum))
  chain <- character(n)
  chain[1] <- 'S'
```



```

U <- runif(n - 1)
for (k in 2:n) {
  etat_actuel <- chain[k - 1]
  chain[k] <- etats[sum(U[k - 1] >
                        P_cum[etats == etat_actuel, ]) + 1]
}
return(chain)
}

```

Afin de vérifier la validité du code, on peut dans un premier temps vérifier si l'on colle bien à la distribution stationnaire. On sait qu'elle existe car nous avons une chaîne de Markov irréductible sur un espace d'états fini.

Voici un code R qui permet de calculer la distribution stationnaire de notre chaîne de Markov.

```

ptrans <- c(0.172, 0.51, 0.33, 0.165)
P <- matrix(c(
  1 - ptrans[1], ptrans[1], 0,
  ptrans[2], 1 - (ptrans[2] + ptrans[3]), ptrans[3],
  0, ptrans[4], 1 - ptrans[4]
), nrow = 3, byrow = TRUE)

distribution_stationnaire <- function(P) {
  res <- eigen(t(P))
  idx <- which.max((res$values))
  dist <- (res$vectors[, idx])
  dist / sum(dist)
}

stat_dist <- distribution_stationnaire(P)
print(stat_dist)

```

On trouve $stat_dist = (0.4970760, 0.1676413, 0.3352827)$ comme mesure stationnaire.

```

err <- matrix(NA, ncol = 3, nrow = 1000)
for(i in 1:1000){
  meteo <- markov(10^4, params)
  err[i, ] <- ((c(sum(meteo == 'S'),
    sum(meteo == 'N'), sum(meteo == 'P'))/10^4) - stat_dist)^2
}
mse <- (1/1000)*colSums(err)

```

Comparée à la version du paragraphe 1.2.2, cette version de *markov* est en moyenne 8 fois plus rapide.

On trouve $mse = (2.2 \times 10^{-4}, 1.3 \times 10^{-5}, 2.3 \times 10^{-4})$

Ce qui semble très satisfaisant mais on peut aller encore plus loin, en comparant cette erreur avec celle commise par un package reconnu.

```

library(markovchain)

ptrans <- c(0.172, 0.51, 0.33, 0.165)
P <- matrix(c(
  1 - ptrans[1], ptrans[1], 0,
  ptrans[2], 1 - (ptrans[2] + ptrans[3]), ptrans[3],
  0, ptrans[4], 1 - ptrans[4]
), nrow = 3, byrow = TRUE)

etats <- c('S', 'N', 'P')

chaine <- new("markovchain",
  states = etats, transitionMatrix = P)

err <- matrix(NA, ncol = 3, nrow = 1000)
for(i in 1:1000){
  meteo <- rmarkovchain(10^4, chaine, t0 = "S")
  err[i, ] <- ((c(sum(meteo == 'S'),
    sum(meteo == 'N'),
    sum(meteo == 'P'))/10^4) - stat_dist)^2
}
mse <- (1/1000)*colSums(err)

```

Et on trouve $mse = (2.5 \times 10^{-4}, 1.3 \times 10^{-5}, 2.5 \times 10^{-4})$

Ceci nous conforte dans le fait que notre simulation est bonne.

4 Calcul de $m(s)$ sous A et B.

4.1 Monte-Carlo simple.

```

msa <- function(n.simul, params) {
  etats <- c(1, 2, 3)
  P <- matrix(c(1 - params$ptrans[1], params$ptrans[1], 0,
    params$ptrans[2],
    1 - (params$ptrans[2] + params$ptrans[3]),
    params$ptrans[3],
    0, params$ptrans[4], 1 - params$ptrans[4]),
    nrow = 3, byrow = TRUE)
  P_cum <- t(apply(P, 1, cumsum))
  all_chain <- numeric(n.simul * 365)

  for(i in 1:n.simul){
    chain <- numeric(365)
    chain[1] <- 1
    U <- runif(364)

    for (k in 2:365) {
      etat_actuel <- chain[k - 1]

```

```

    chain[k] <- etats[sum(U[k - 1] > P_cum[etat_actuel, ]
                        + 1)]
  }

  all_chain[((i - 1) * 365 + 1):(i * 365)] <- chain
}

c1 <- all_chain == 1
c2 <- all_chain == 2
c3 <- all_chain == 3
all_chain[c1] <- params$pacc[1]
all_chain[c2] <- params$pacc[2]
all_chain[c3] <- params$pacc[3]

acc <- rowSums(matrix(rbinom(n.simul*365, params$N,
                           all_chain), nrow = n.simul, ncol = 365))
remb <- numeric(n.simul)
for(i in 1:n.simul) {
  remb[i] <- sum(rremb(acc[i], params))
}

X <- remb[remb > params$s]

m <- mean(X)
Q <- length(X > params$s)

sig <- sqrt((1/(Q - 1)) * sum((X - m)^2))
u <- qnorm(0.975)

ms <- m - params$s
demi.largeur <- u*sig/sqrt(Q)
return(list(ms = ms, demi.largeur = demi.largeur))
}

```

Pour msb, on va directement utiliser la fonction Markov et on va implémenter une fonction accB qui permet de calculer le nombre d'accidents sur n années consécutives, sous l'hypothèse **B**.

```

accB <- function(n, params) {
  acc_tot <- numeric(n)
  for(i in 1:n){
    m <- replicate(params$N, markov(365, params))
    meteo_sim <- colSums(matrix(
      c(colSums(m == 'S'), colSums(m == 'N'), colSums(m == 'P')),
      ncol = 3
    ))
    acc <- rbinom(3, meteo_sim, params$pacc)
    acc_tot[i] <- sum(acc)
  }
  acc_tot
}

```

```

}

msb <- function(n.simul, params) {
  acc <- accB(n.simul, params)
  remb <- sapply(acc, function(u) (sum(rremb(u, params))))
  X <- remb[remb > params$s]

  m <- mean(X)
  N <- length(X)
  sig <- sqrt((1/(N - 1)) * sum((X - m)^2))
  ms <- m - params$s
  u <- qnorm(0.975)
  return(list(ms = ms, demi.largeur = u*sig/sqrt(N)))
}

```

4.2 Monte-Carlo avec réduction de variance.

La méthode employée sera celle de la réduction de variance par variable de contrôle. On prend $r_i \sim \mathcal{N}(e, v)$ avec e , v respectivement l'espérance et la variance d'un remboursement selon la densité f et les $(r_i)_i$ iid.

On pose Z la variable aléatoire représentant le nombre total de remboursement à l'année. Connaissant le nombre d'accidents acc sur l'année, on sait que $Z \sim \mathcal{N}(acc \times e, v \times acc)$.

Et donc

$$R_a = R - k(Z - \mu), \quad k = \frac{\text{cov}(R, Z)}{\text{var}(Z)}$$

Et on travaillera sur ce R_a en espérant que la variance des résultats soient réduites.

Pour rappeller, les constantes c , e , v ont été calculées précédemment.

```

msa <- function(n.simul, params) {
  etats <- c(1, 2, 3)
  P <- matrix(c(1 - params$ptrans[1], params$ptrans[1], 0,
    params$ptrans[2],
    1 - (params$ptrans[2] + params$ptrans[3]),
    params$ptrans[3],
    0, params$ptrans[4], 1 - params$ptrans[4]),
    nrow = 3, byrow = TRUE)
  P_cum <- t(apply(P, 1, cumsum))
  all_chain <- numeric(n.simul * 365)

  for(i in 1:n.simul){
    chain <- numeric(365)
    chain[1] <- 1
    U <- runif(364)

    for (k in 2:365) {

```

```

    etat_actuel <- chain[k - 1]
    chain[k] <- etats[sum(U[k - 1] > P_cum[etat_actuel, ])
                      + 1]
  }

  all_chain[((i - 1) * 365 + 1):(i * 365)] <- chain
}

c1 <- all_chain == 1
c2 <- all_chain == 2
c3 <- all_chain == 3
all_chain[c1] <- params$pacc[1]
all_chain[c2] <- params$pacc[2]
all_chain[c3] <- params$pacc[3]

acc <- rowSums(matrix(rbinom(n.simul*365, params$N,
                           all_chain), nrow = n.simul, ncol = 365))
remb <- numeric(n.simul)
for(i in 1:n.simul) {
  remb[i] <- sum(rremb(acc[i], params))
}

means <- e*acc
sds <- sd*acc
Z <- rnorm(n.simul, means, sds)
k <- cov(remb, Z)/(sds^2)

X <- remb - k*(Z - means)
X <- X[X > params$s]

m <- mean(X)
Q <- length(X > params$s)

sig <- sqrt( (1/(Q - 1)) * sum((X - m)^2) )
u <- qnorm(0.975)

ms <- m - params$s
demi.largeur <- u*sig/sqrt(Q)
return(list(ms = ms, demi.largeur = demi.largeur) )
}

```

Afin de vérifier si la réduction de variance a un intérêt, on peut par exemple effectuer un *grand* nombre de fois msa avec et sans réduction de variance et voir les effets que cela a sur les résultats finaux.

On a modifié msa dans le return :

```

msa <- function(n.simul, params){
  ...
  return(c(ms, demi.largeur))
}

```

```

}

A <- replicate(100, msa(5000, params))
B <- replicate(100, msa2(5000, params))

reduc_v <- c()
for(i in 1:2){
  reduc_v[i] <- ((var(B[i,]) - var(A[i,]))/var(B[i,]))*100
}
print(reduc_v)

```

Et on trouve environ $reduc_v = (19, 29)$, ce qui veut dire que l'on a réduit la variance des résultats obtenus sur ms d'environ 20% et sur la demi-largeur d'environ 30%.

Pour la simplicité de l'implémentation, nous implémenterons cette méthode de réduction de variance toutes les autres fonctions, de l'exacte même façon.

4.3 Implémentation de msb.c

Pour l'implémentation de msb en C++, on utilisera le package *Rcpp* de R-Studio.

On a choisi d'implémenter accB en C++ car la nature du langage C++ (compilé) permet de gagner en rapidité d'exécution surtout au niveau des boucles. Ainsi, dans la fonction accB_cpp on fait toutes les chaînes de Markov dont nous avons besoin car il y a beaucoup de boucles inévitables qui ralentissent le code en R. Nous avons aussi rajouter une fonction rremb_sum qui permet de calculer la somme des remboursements, simplement car elle était un peu plus rapide que la version en R.

Notons aussi que nous avons décider de conserver une partie du code en R, simplement car msb perd du temps quasi exclusivement sur les boucles, le reste des fonctions employées sont nativement codées en C dans R et ne devraient donc pas faire perdre de temps par rapport à une implémentation en C++.

```

msb.c <- function(n.simul, params) {
  library(Rcpp)
  cppFunction(
    NumericVector rremb_sum(IntegerVector ns, List params) {
      RNGScope scope;

      NumericVector result(ns.size());

      double eta = as<double>(params["eta"]);
      double alpha = as<double>(params["alpha"]);
      double x0 = as<double>(params["x0"]);

      double e = 1 - eta;
      double a = alpha + x0;
    }
  )
}

```

```

double a_m = x0 - alpha;
double c=(1-eta)/(pow(alpha+x0,e)-2*pow(alpha,e));
double F_x0 = c * (pow(alpha, e) / e) + 1;
double inv_e = 1 / e;
double a_e = pow(a, e);
double e_c = e / c;

for (int j = 0; j < ns.size(); j++) {
    int n = ns[j];
    NumericVector y = runif(n, 0, 1);

    NumericVector sum_result(n);
    for (int i = 0; i < n; i++) {
        if (y[i] < F_x0) {
            sum_result[i]=a-exp(inv_e*log(a_e-e_c*y[i]));
        } else {
            sum_result[i]=a_m+exp(inv_e*log(e_c*(y[i]-1)));
        }
    }

    result[j] = sum(sum_result);
}

return result;
})
cppFunction(
IntegerVector accB_cpp(int n, List params) {
    RNGScope scope;
    int N = params["N"];
    NumericVector ptrans = params["ptrans"];
    IntegerVector states = {1, 2, 3};
    NumericMatrix P(3, 3);
    IntegerVector acc(n);
    IntegerVector acc2(N);

    NumericVector pacc = params["pacc"];
    double Sol = pacc[0];
    double Nua = pacc[1];
    double Plu = pacc[2];

    P(0, 0) = 1 - ptrans[0]; P(0, 1) = ptrans[0];
    P(0, 2) = 0;
    P(1, 0) = ptrans[1];
    P(1, 1) = 1 - (ptrans[1] + ptrans[2]);
    P(1, 2) = ptrans[2];
    P(2,0)=0;P(2,1)=ptrans[3];P(2,2)=1-ptrans[3];

    for (int i = 0; i < n; i++) {
        IntegerVector chain(365);
        chain[0] = 1;

```

```

NumericVector p_acc(365);
p_acc[0] = Sol;
for (int r = 0; r < N; r++){
  for (int k = 1; k < 365; k++) {
    int current_state = chain[k - 1] - 1;
    NumericVector probs(3);
    for (int j = 0; j < 3; j++) {
      probs[j] = P(current_state, j);
    }
    chain[k] = Rcpp::sample(states, 1, true, probs)[0];
    if (chain[k] == 1) {
      p_acc[k] = Sol;
    } else if (chain[k] == 2) {
      p_acc[k] = Nua;
    } else {
      p_acc[k] = Plu;
    }
  }
}

NumericVector results(365);
for (int l = 0; l < 365; l++) {
  results[l] = R::rbinom(1, p_acc[l]);
}
acc2[r] = sum(results);
}

acc[i] = sum(acc2);
}

return acc;
}
)

acc <- accB_cpp(n.simul, params)

remb <- rremb_sum(acc, params)
means <- e*acc
sds <- sd*sqrt(acc)

Z <- rnorm(n.simul, means, sds)
k <- cov(remb, Z)/(sds^2)

X <- remb - k*(Z - means)
X <- X[X > params$s]

m <- mean(X)
N <- length(X)
sig <- sqrt( (1/(N - 1)) * sum((X - m)^2) )
ms <- m - params$s
u <- qnorm(0.975)

```



```

    return(list(ms = ms, demi.largeur = u*sig/sqrt(N)))
}

```

Gain de temps Comme escompté, msb.c est plus rapide que msb, d'un facteur 8 environ.

5 Pistes d'améliorations.

5.1 Le modèle.

Le modèle ne tient pas compte du fait de l'âge des assurés, les jeunes conducteurs sont beaucoup plus susceptible d'avoir un accidents que les conducteurs plus âgés. Il faudrait donc par exemple faire deux classes d'assurés avec des probabilités d'accidents différentes selon la classe. Une classe "Jeune conducteur" et une classe "Conducteur confirmé" avec des probabilités d'accidents plus élevés pour la classe "Jeune conducteur".

Ensuite, le modèle estime que les montants remboursés sont uniformes sur tous les accidents, or on pourrait avoir différents remboursements en fonction de la météo, par exemple par temps sec les accidents sont plus graves¹.

5.2 Le programme.

On sait que $\mathbb{P}(R > s)$ est petit (empiriquement, sous l'hypothèse **A** on constate que cette probabilité est proche de 5%).

Ainsi, on essaie de faire une Monte-Carlo sur assez peu d'échantillons, car sur 10 000 ans, seulement 500 à 600 années portent un remboursement total supérieur au seuil. Et la nature du programme fait qu'il serait nécessaire de le laisser tourner très longtemps pour obtenir un nombre d'échantillons suffisant pour faire une Monte-Carlo fiable.

Ainsi, une piste d'amélioration serait de résoudre ce problème, par exemple en modifiant/pondérant la densité de sorte à artificiellement favoriser des remboursements totaux qui dépasseront le seuil. Nous n'avons malheureusement par réussi à obtenir quelque chose de probant et juste mathématiquement.

Références

- [1] **MétéoFrance** Disponible à : <https://meteofrance.com/comprendre-climat/france/le-climat-en-france-metropolitaine>. Consulté le 25 novembre 2024.
- [2] **Actualitix** Disponible à : <https://www.actualitix.com/nombre-d-heures-d-enseillement-par-departement.html>. Consulté le 25 novembre 2024.

1. <https://www.awsr.be/les-accidents-a-moto-2-fois-plus-graves-quand-il-fait-beau/>

- [3] **DDG** Disponible à : <https://www.statistiques.developpement-durable.gouv.fr/sites/default/files/2018-11/lps156-deux-roues-motorises.pdf>. Consulté le 14 décembre 2024.
- [4] **DDG2** Disponible à : <https://www.statistiques.developpement-durable.gouv.fr/donnees-sur-le-parc-automobile-francais-au-1er-janvier-2024>. Consulté le 14 décembre 2024.
- [5] **Tribune Auto** Disponible à : <https://www.latribuneauto.com/reportages/actualites/15074-la-voiture-occupe-une-place-preponderante-dans-les-trajets-domicile-travail-des-actifs-francais-en-2024>. Consulté le 14 décembre 2024.
- [6] **Lefigaro** Disponible à : <https://www.lefigaro.fr/assurance/2012/03/06/05005-20120306ARTFIG00767-accident-automobile-tout-savoir-sur-le-constat-amiable.php>. Consulté le 14 décembre 2024.